

Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 3 - Semantic Analysis

Semester II tahun 2025/2026



Oleh:

Kelompok ReguExceptional:

Wafiq Hibban Robbany	13524016
Muhammad Nafis Habibi	13524018
An-Dafa Anza Avansyah	13524038
Wildan Abdurrahman Ghazali	13524054

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

21 Mei 2026

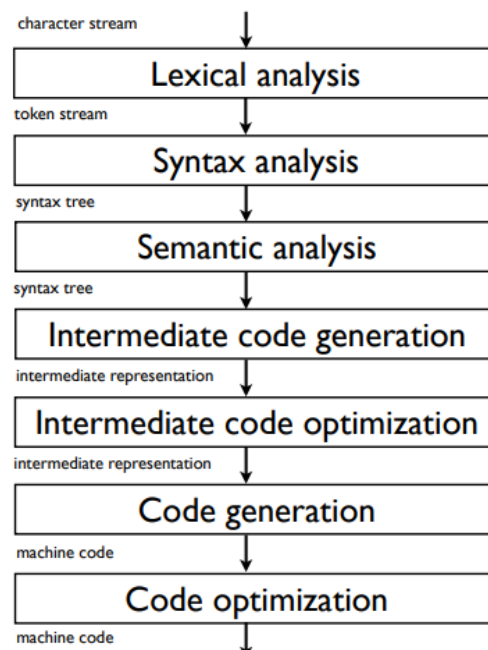
Daftar Isi

Daftar Isi.....	2
Dasar Teori.....	3
A. Semantic Analysis.....	3
B. Abstract Syntax Tree (AST).....	5
C. Decorated Abstract Syntax Tree (Decorated AST).....	7
Perancangan dan Implementasi.....	10
A. AST.....	10
a. ASTNode.....	10
b. ASTBuilder.....	14
B. Decorated AST.....	19
a. Symbol Table.....	19
b. Pembuatan Decorated AST.....	21
Pengujian.....	25
Kesimpulan dan Saran.....	27
A. Kesimpulan.....	27
B. Saran.....	27
Lampiran.....	28
Referensi.....	29

Dasar Teori

A. Semantic Analysis

Analisis semantik merupakan tahapan krusial dalam proses kompilasi yang bertugas untuk memeriksa keabsahan makna atau konsistensi logis dari suatu kode sumber terhadap definisi bahasa pemrograman yang digunakan. Meskipun tahapan analisis sintaksis sebelumnya telah berhasil memastikan bahwa urutan token memenuhi aturan tata bahasa dan menghasilkan *parse tree*, *Context-Free Grammar* tidak mampu merepresentasikan seluruh batasan bahasa, terutama hubungan yang bersifat non-lokal atau bergantung pada konteks. Sebagai contoh, sebuah *grammar* dapat menyatakan bahwa struktur penugasan nilai adalah valid secara tata bahasa, tetapi ia tidak bisa mendeteksi apakah variabel yang dituju sudah dideklarasikan atau apakah tipe datanya cocok. Oleh karena itu, tahapan analisis semantik menjadi fase kompilator terakhir yang dapat mendeteksi dan melaporkan kesalahan-kesalahan logis tersebut kepada pengguna sebelum kode didekonstruksi ke tahapan penerjemahan selanjutnya.

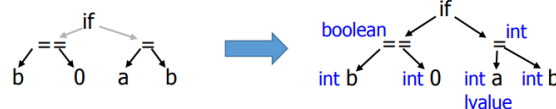


Gambar 1. Struktur Tahapan Kompilator di dalam Alur Kompilasi

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>

Secara umum, pengecekan yang dilakukan pada tahapan analisis semantik mencakup beberapa validasi statis yang mendasar. Pertama adalah pengecekan tipe (*type checking*) yang berfungsi menjamin bahwa setiap operator maupun fungsi diterapkan pada operan dengan tipe data dan jumlah argumen yang kompatibel atau sesuai dengan aturan bahasa, seperti mencegah operasi aritmatika antara tipe bilangan dengan tipe teks. Kedua adalah manajemen tabel simbol (*symbol table management*) untuk mengelola deklarasi *identifier*. Proses ini memastikan bahwa suatu variabel atau fungsi tidak dapat digunakan sebelum dideklarasikan terlebih dahulu, serta mendeteksi apabila terdapat penanda yang dideklarasikan secara ganda dalam satu lingkup yang sama. Ketiga adalah resolusi lingkup (*scope resolution*) untuk menentukan validitas aksesibilitas suatu penanda berdasarkan hierarki blok kode tempat ia berada, seperti membedakan variabel lingkup global dan lokal. Keempat adalah validasi alur kontrol (*control flow validation*) untuk memastikan kelogisan struktur instruksi eksekusi, seperti memastikan perintah penghentian perulangan hanya diletakkan di dalam blok *loop* yang valid.

Input: syntax tree $\Rightarrow$ **Output:** decorated syntax tree



Gambar 2. Perubahan Representasi dari *Syntax Tree* Menjadi *Decorated Syntax Tree*

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>

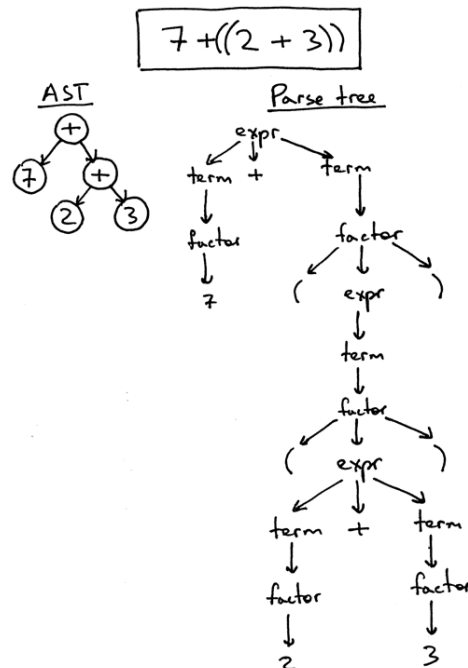
Dalam eksekusinya, analisis semantik memanfaatkan metode formal berupa *Syntax-Directed Definition* (SDD) atau *Attributed Grammar*, yaitu aturan tata bahasa bebas konteks yang dilengkapi dengan atribut semantik dan aturan kalkulasi pada setiap produksi *grammar*-nya. Penelusuran pohon sintaksis dilakukan secara *top-down* atau *Depth-First Search* (DFS) menggunakan skema *L-Attributed Grammar* melalui pemanggilan fungsi penelusuran (*visitor pattern*). Atribut dalam skema ini dievaluasi secara teratur, di mana atribut turunan dihitung saat penelusuran bergerak ke bawah, sedangkan atribut sintesis dihitung saat bergerak kembali ke atas pohon. Seluruh informasi penanda yang ditemui

selama penelusuran disimpan ke dalam struktur data tabel simbol yang berbasis *stack*. Pada arsitektur kompilator bahasa Arion, sistem manajemen penanda ini dikelola secara ketat melalui pembagian tiga jenis tabel terpisah dengan tanggung jawab spesifik, yaitu tabel *tab* untuk menyimpan detail informasi umum seluruh *identifier* (seperti konstanta, variabel, prosedur, atau fungsi), tabel *btabs* untuk merepresentasikan blok prosedur atau definisi tipe *record*, dan tabel *atab* untuk mencatat spesifikasi batasan indeks serta elemen dari struktur data *array*.

B. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah representasi pohon (*tree*) dari struktur sintaksis abstrak sebuah kode sumber yang ditulis dalam suatu bahasa pemrograman. Dalam konsep kompilasi, AST pada dasarnya adalah bentuk ringkas dari *Parse Tree*. Pada AST, sistem hanya menyimpan informasi krusial yang diperlukan untuk tahapan kompilasi selanjutnya dan membuang informasi sintaksis yang sudah tidak lagi diperlukan, seperti tanda kurung pengelompokan, kata kunci penanda blok, atau titik koma. Kata "abstrak" pada AST merujuk pada sifatnya yang tidak lagi merepresentasikan setiap detail gramatikal yang muncul pada sintaksis kode aslinya. Sebaliknya, AST berfokus pada aturan semantik, yaitu mengenai hubungan logis antara operator dan operannya.

Perbedaan mendasar antara AST dan *Parse Tree* terletak pada tingkat kedetailan gramatikalnya. Struktur dasar sebuah *Parse Tree* dibangun dengan sangat ketat berdasarkan aturan *Grammar* murni untuk tujuan pengecekan sintaksis (*syntax checking*). Setiap simbol terminal dan non-terminal divisualisasikan sebagai simpul (*node*). Namun, pada tahap *Semantic Analysis* dan pemrosesan selanjutnya, konteks gramatikal yang terlalu mendetail tersebut tidak lagi diperlukan.

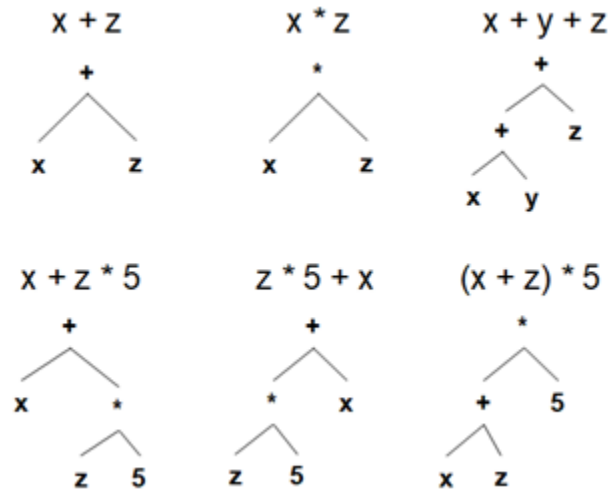


Gambar 3. Perbedaan Parse Tree dan AST pada Ekspresi Matematika

<https://ruslanspivak.com/lbasi-part7/>

Sebagai contoh dari perbedaan tersebut, konstruksi blok percabangan pada *Parse Tree* mungkin membutuhkan banyak *node* terminal pelengkap, sedangkan pada AST, blok tersebut dapat diringkas menjadi sebuah *node* tunggal yang langsung memiliki cabang untuk kondisi dan blok eksekusi lanjutannya.

Untuk memastikan AST dapat diproses dengan benar oleh tahap selanjutnya, terdapat beberapa persyaratan inti dalam merepresentasikan informasi di dalam pohon abstrak ini. Pertama, urutan dari pernyataan yang akan dieksekusi harus direpresentasikan secara eksplisit dan terdefinisi dengan baik. Kedua, untuk operasi biner, komponen operan sebelah kiri dan sebelah kanan harus disimpan serta diidentifikasi dengan benar sesuai posisinya, mengingat ekspresi pada umumnya dievaluasi secara *left-associative*. Ketiga, *identifier* beserta nilai yang ditugaskan kepadanya harus disimpan dengan jelas di dalam pohon, terutama untuk pernyataan *assignment*.



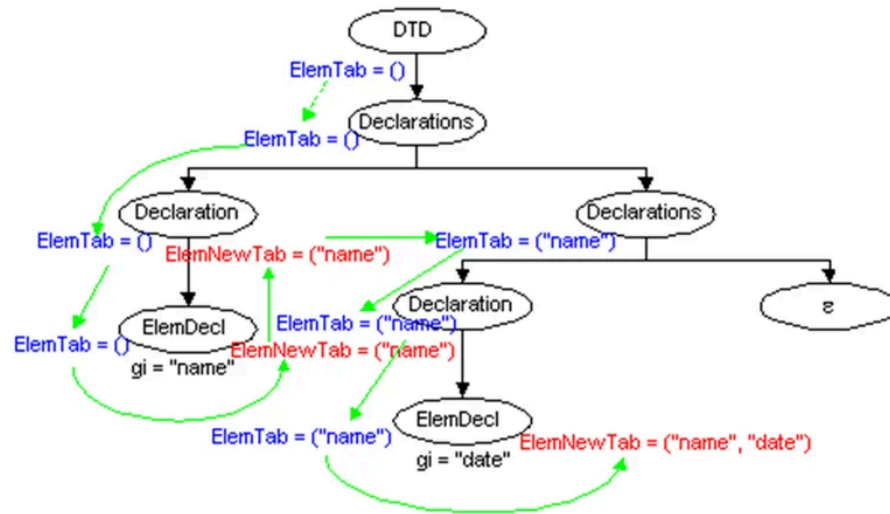
Gambar 4. Contoh AST untuk Berbagai Ekspresi Biner

<https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>

Secara hierarkis, *node* di dalam AST secara tipikal berkorelasi dengan *non-terminal* dari aturan sintaksis, di mana *node* non-daun mewakili sisi kiri dari aturan produksi, dan anak-anaknya (*children*) mewakili sisi kanan. Untuk membedakan fungsinya, *node* pada AST umumnya diberi label yang merepresentasikan nilai dan tipe sintaksisnya. Melalui hierarki *node* ini, pengelompokan operasi menjadi tersirat di dalam struktur kedalaman cabangnya, sehingga menghasilkan representasi memori yang jauh lebih efisien untuk dievaluasi oleh *compiler*.

C. Decorated Abstract Syntax Tree (Decorated AST)

Decorated Abstract Syntax Tree atau *Decorated AST* adalah bentuk lanjutan dari *Abstract Syntax Tree* (AST) yang telah diberi informasi tambahan hasil proses *semantic analysis*. *Decorated AST* merepresentasikan struktur semantik program (AST) ditambah anotasi yang dibutuhkan untuk memastikan bahwa program memiliki makna yang valid. Anotasi ini dapat berupa tipe data, referensi ke entri *symbol table*, *lexical level*, informasi *scope*, status deklarasi, atau informasi lain yang dibutuhkan untuk pengecekan semantik. Setelah proses *semantic analysis*, sebuah *node* pada AST dapat didekorasi menjadi tipe variabel tertentu, berada pada *lexical level* tertentu, dan memiliki indeks tertentu pada *symbol table*.



Gambar 5. Contoh Decorated Abstract Syntax Tree

https://www.researchgate.net/figure/Figura-1-Decorated-Abstract-Syntax-Tree_fig1_237825111

Pada proses *semantic analysis*, *Decorated AST* dibangun dengan menelusuri AST menggunakan fungsi *visitor*. Setiap jenis node memiliki fungsi kunjungan tertentu, misalnya *visitAssignNode*, *visitVarNode*, *visitBinOpNode*, dan *visitIfNode*. Fungsi-fungsi tersebut bertugas memanggil visit pada *child node*, mengakses atau memperbarui *symbol table*, menghitung tipe hasil ekspresi, serta menambahkan anotasi pada node.

Symbol table merupakan struktur data utama yang digunakan dalam proses dekorasi AST yang menyimpan informasi identifier yang muncul dalam program, seperti nama variabel, konstanta, tipe, prosedur, fungsi, parameter, atau *field record*. Informasi ini digunakan untuk mengecek apakah suatu identifier sudah dideklarasikan, apakah terjadi deklarasi ulang dalam scope yang sama, serta tipe apa yang dimiliki oleh identifier tersebut. *Symbol table* terdiri dari tiga tabel, yaitu *tab*, *btab*, dan *atab*. *tab* digunakan untuk menyimpan informasi identifier seperti nama, jenis objek, tipe, referensi, lexical level, dan alamat. *btab* digunakan untuk menyimpan informasi block, seperti block prosedur, fungsi, atau record. *atab* digunakan untuk menyimpan informasi array, seperti tipe indeks, batas bawah, batas atas, tipe elemen, dan ukuran array.

Dalam proses pembentukan Decorated AST, *semantic analyzer* melakukan *type checking* dan *scope checking*. *Type checking* bertujuan memastikan bahwa setiap operasi, ekspresi, assignment, dan pemanggilan prosedur atau fungsi menggunakan tipe data yang sesuai. Misalnya, operasi aritmatika harus dilakukan pada operand numerik, operasi logika harus menggunakan operand boolean, dan nilai yang diberikan pada assignment harus kompatibel dengan tipe variabel tujuan. Sementara itu, *scope checking* bertujuan memastikan bahwa setiap identifier digunakan pada lingkup yang valid. Identifier yang digunakan dalam ekspresi atau statement harus sudah dideklarasikan sebelumnya, baik pada scope lokal maupun scope luar yang masih dapat diakses.

Perancangan dan Implementasi

Implementasi dilakukan dalam bahasa C++ dan dibuat dalam *namespace* "arion" agar terpisah dari *standard library* C++.

A. AST

a. ASTNode

Implementasi `ASTNode` digunakan sebagai struktur utama untuk merepresentasikan simpul pada Abstract Syntax Tree. Berbeda dengan *parse tree* yang masih menyimpan seluruh detail *grammar*, AST hanya menyimpan informasi yang penting untuk tahap *semantic analysis* dan tahap kompilasi berikutnya. Oleh karena itu, node-node seperti tanda kurung, titik koma, keyword `begin`, `end`, `then`, `do`, dan terminal sintaksis lain yang tidak membawa makna semantik tidak dimasukkan lagi ke dalam AST.

Pada implementasi ini, setiap node AST memiliki jenis atau kategori yang direpresentasikan melalui enum `ASTNodeKind`. Enum ini mendefinisikan berbagai bentuk node yang dibutuhkan oleh bahasa Arion, seperti `Program`, `Declarations`, `ConstDeclaration`, `VarDeclaration`, `ProcedureDeclaration`, `FunctionDeclaration`, `Assignment`, `IfStatement`, `WhileStatement`, `ForStatement`, `BinaryOperation`, `UnaryOperation`, `Variable`, `ArrayAccess`, `FieldAccess`, serta berbagai literal seperti `IntegerLiteral`, `RealLiteral`, `CharLiteral`, `StringLiteral`, dan `BooleanLiteral`. Dengan penggunaan enum tersebut, setiap simpul AST dapat dikenali berdasarkan perannya dalam struktur program, bukan lagi berdasarkan nama *non-terminal* grammar.

Selain jenis *node*, AST juga menyimpan relasi antar-*node* melalui konsep *child role* yang direpresentasikan oleh enum `ASTChildRole`. *Role* ini digunakan untuk menjelaskan posisi atau fungsi sebuah *child* terhadap *parent*-nya. Sebagai contoh, pada *node assignment*, *child* dengan *role* `Target` merepresentasikan variabel tujuan, sedangkan *child* dengan *role* `Value` merepresentasikan ekspresi yang diberikan. Pada *node* operasi biner, *role* `Left` dan `Right` digunakan untuk membedakan operand kiri dan kanan. Pada

node percabangan, role Condition, Then, dan Else digunakan untuk menyimpan kondisi, blok ketika kondisi benar, dan blok alternatif jika ada.

Setiap ASTNode juga dapat menyimpan atribut tambahan dalam bentuk pasangan nama dan nilai. Atribut ini digunakan untuk menyimpan informasi sederhana seperti nama identifier, nama program, operator, atau nilai literal. Misalnya, *node* Variable dapat memiliki atribut name, sedangkan *node* IntegerLiteral dapat memiliki atribut value. Dengan cara ini, informasi penting dari token tetap dipertahankan, tetapi tanpa membawa seluruh struktur *parse tree* yang terlalu rinci.

Struktur AST juga disiapkan agar dapat didekorasi pada tahap semantic analysis. Hal ini ditunjukkan melalui adanya struktur anotasi yang menyimpan informasi semantik seperti nama tipe, indeks pada symbol table, indeks block, indeks array, dan lexical level. Informasi ini belum tentu langsung lengkap saat AST pertama kali dibentuk, tetapi akan diisi pada tahap berikutnya ketika *semantic analyzer* melakukan pengecekan tipe, pengecekan deklarasi, dan resolusi scope. Dengan demikian, ASTNode tidak hanya berfungsi sebagai representasi sintaksis abstrak, tetapi juga sebagai dasar pembentukan Decorated AST.

Berikut adalah struktur data AST yang terdapat pada src/arion/AST.hpp:

```
namespace arion {  
    struct ASTChild;  
  
    enum class ASTNodeKind {  
        Program,  
        Declarations,  
        ConstDeclarations,  
        ConstDeclaration,  
        TypeDeclarations,  
        TypeDeclaration,  
        VarDeclarations,  
        VarDeclaration,  
        FieldDeclaration,  
        ProcedureDeclaration,  
        FunctionDeclaration,  
        Parameters,  
    };
```

```
ParameterGroup,  
Parameter,  
Block,  
CompoundStatement,  
StatementList,  
EmptyStatement,  
Assignment,  
IfStatement,  
CaseStatement,  
CaseBranch,  
WhileStatement,  
RepeatStatement,  
ForStatement,  
ProcedureCall,  
FunctionCall,  
Arguments,  
BinaryOperation,  
UnaryOperation,  
Variable,  
ArrayAccess,  
FieldAccess,  
IntegerLiteral,  
RealLiteral,  
CharLiteral,  
StringLiteral,  
BooleanLiteral,  
NamedType,  
ReturnType,  
ArrayType,  
RecordType,  
RangeType,  
EnumeratedType,  
Identifier,  
Unknown  
};  
  
enum class ASTChildRole {  
    None,  
    Declaration,  
    Const,  
    Type,  
    Variable,  
    Field,  
    Parameter,  
    Group,  
    Parameters,  
    ReturnType,  
    Block,  
};
```

```

        Body,
        Statement,
        Target,
        Value,
        Condition,
        Then,
        Else,
        Expression,
        Branch,
        Label,
        Left,
        Right,
        Base,
        Index,
        Element,
        Low,
        High,
        Arg,
        Start,
        End
    };

    struct ASTAnnotation {
        std::string typeName;
        int tabIndex = -1;
        int blockIndex = -1;
        int arrayIndex = -1;
        int lexicalLevel = -1;
    };

    class ASTNode {
    public:
        explicit ASTNode(ASTNodeKind kind =
ASTNodeKind::Unknown, int line = -1);

        ASTNode &addChild(ASTNode child);
        ASTNode &addChild(ASTChildRole role, ASTNode child);
        ASTNode &setAttribute(std::string key, std::string
value);

        ASTNodeKind getKind() const;
        std::string getAttribute(const std::string &key)
const;

        const std::vector<ASTChild> &getChildren() const;
        const std::vector<std::pair<std::string, std::string>>
&getAttributes() const;
        const ASTNode &childAt(std::size_t index) const;
        ASTNode &childAt(std::size_t index);
    };

```

```

        const ASTNode *childWithRole(ASTChildRole role) const;
        ASTNode *childWithRole(ASTChildRole role);
        std::vector<const ASTNode
*>childrenWithRole(ASTChildRole role) const;
        std::vector<ASTNode *> childrenWithRole(ASTChildRole
role);

        void setAnnotation(ASTAnnotation annotation);
        ASTAnnotation &annotation();
        const ASTAnnotation &annotation() const;

        ASTNode & setLine(int line);
        int getLine() const;

        static std::string kindToString(ASTNodeKind kind);
        static std::string roleToString(ASTChildRole role);

        void printTree(std::ostream& out = std::cout) const;

    private:
        ASTNodeKind kind_;
        int line_;
        ASTAnnotation annotation_;
        std::vector<std::pair<std::string, std::string>>
attributes_;
        std::vector<ASTChild> children_;

        void printTreeHelper(int depth, std::vector<bool>
&isLast, ASTChildRole role, std::ostream& out = std::cout)
const;
    };

    struct ASTChild {
        ASTChildRole role;
        ASTNode node;
    };
}

```

b. ASTBuilder

ASTBuilder merupakan komponen yang bertanggung jawab untuk mengubah *parse tree* hasil *parser* menjadi Abstract Syntax Tree. Proses ini dilakukan dengan menelusuri *node parse tree*, mengenali label *non-terminal* tertentu, kemudian membentuk node AST yang lebih ringkas dan bermakna secara semantik. Fungsi utama yang

digunakan adalah *build*, yang menerima akar *parse tree* dan meneruskan proses pembentukan AST ke fungsi internal *buildNode*.

Fungsi *buildNode* bekerja sebagai *dispatcher*. Fungsi ini membaca label dari sebuah *ParseNode*, kemudian memanggil fungsi pembangun yang sesuai dengan jenis *non-terminal* tersebut. Sebagai contoh, label *<program>* akan diarahkan ke *buildProgram*, label *<declaration-part>* diarahkan ke *buildDeclarationPart*, label *<var-declaration>* diarahkan ke *buildVarDeclaration*, label *<assignment-statement>* diarahkan ke *buildAssignmentStatement*, dan label *<expression>* diarahkan ke *buildExpression*. Dengan desain ini, proses pembentukan AST menjadi lebih modular karena setiap konstruksi *grammar* ditangani oleh fungsi khusus.

Pada bagian program utama, *buildProgram* membentuk *node* Program. Fungsi ini mengambil nama program dari header program, kemudian menambahkan bagian deklarasi dan *body* program sebagai *child*. Bagian deklarasi direpresentasikan sebagai *node* Declarations, sedangkan bagian *body* direpresentasikan melalui *node* compound statement. Dengan demikian, struktur utama program dalam AST menjadi lebih sederhana, yaitu terdiri atas nama program, kumpulan deklarasi, dan blok eksekusi utama.

Untuk deklarasi, *ASTBuilder* menyediakan beberapa fungsi khusus seperti *buildConstDeclaration*, *buildTypeDeclaration*, dan *buildVarDeclaration*. Pada deklarasi konstanta, builder mengambil nama konstanta dan nilai constant yang bersesuaian, lalu membentuk *node* ConstDeclaration. Pada deklarasi tipe, builder mengambil nama tipe dan definisi tipenya, lalu membentuk *node* TypeDeclaration. Pada deklarasi variabel, builder membaca daftar identifier dari *<identifier-list>*, kemudian memasang setiap identifier dengan tipe yang sama. Jika terdapat deklarasi seperti a, b: Integer, maka *ASTBuilder* akan membentuk dua *node* VarDeclaration, masing-masing untuk a dan b, dengan *child* bertipe Integer.

Untuk tipe data, *ASTBuilder* membentuk *node* yang sesuai dengan jenis tipe yang ditemukan. Tipe bernama seperti Integer, Real, atau tipe buatan pengguna akan dibentuk

sebagai NamedType. Tipe array dibentuk sebagai ArrayType dengan *child* untuk *index* dan *element type*. Tipe subrange dibentuk sebagai RangeType dengan *child* Low dan High. Tipe enumerasi dibentuk sebagai EnumeratedType yang berisi daftar elemen identifier. Sementara itu, tipe record dibentuk sebagai RecordType yang memiliki *child* berupa deklarasi *field*. Pendekatan ini membuat struktur tipe pada AST lebih mudah dianalisis oleh *semantic analyzer*, terutama saat membangun *symbol table* dan melakukan *type checking*.

Untuk statement, builder menyediakan fungsi seperti buildCompoundStatement, buildStatementList, buildStatement, buildAssignmentStatement, buildIfStatement, buildWhileStatement, buildRepeatStatement, dan buildForStatement. Compound statement dibentuk sebagai *node* CompoundStatement yang memiliki *child* berupa daftar statement. Statement list dibentuk sebagai *node* StatementList yang menyimpan setiap statement secara berurutan. Jika suatu statement kosong, builder menghasilkan *node* EmptyStatement, sehingga program dengan statement kosong tetap dapat direpresentasikan secara valid.

Pada assignment statement, ASTBuilder membentuk *node* Assignment. Node ini memiliki *child* Target yang berasal dari variable dan *child* Value yang berasal dari expression. Sebagai contoh, statement $x := x + 1$ akan direpresentasikan sebagai *node assignment* dengan target x dan value berupa operasi biner penjumlahan. Dengan representasi seperti ini, *semantic analyzer* dapat dengan mudah memeriksa apakah variabel tujuan sudah dideklarasikan dan apakah tipe ekspresi kompatibel dengan tipe variabel tersebut.

Pada expression, *builder* menyusun struktur operasi berdasarkan operator dan operand. Operasi biner direpresentasikan sebagai *node* BinaryOperation dengan atribut *operator* serta *child* Left dan Right. Operasi *unary* direpresentasikan sebagai *node* UnaryOperation. Literal angka, karakter, string, dan boolean dibentuk sebagai *node* literal yang sesuai, seperti IntegerLiteral, RealLiteral, CharLiteral, StringLiteral, atau

BooleanLiteral. Identifier yang digunakan sebagai variabel dibentuk sebagai node Variable dengan atribut name.

Selain statement dan expression, ASTBuilder juga menangani pemanggilan prosedur dan fungsi. Pemanggilan prosedur direpresentasikan sebagai ProcedureCall, sedangkan pemanggilan fungsi direpresentasikan sebagai FunctionCall. Argumen pemanggilan disimpan dalam *node* Arguments, sehingga *semantic analyzer* dapat melakukan pengecekan jumlah dan tipe parameter terhadap deklarasi prosedur atau fungsi yang tersimpan di *symbol table*.

Berikut adalah kelas ASTBuilder yang terdapat pada src/arion/ASTBuilder.hpp:

```
namespace arion {
    class ASTBuilder {
    public:
        ASTNode build(const ParseNode &parseTree) const;

    private:
        ASTNode buildNode(const ParseNode &node) const;
        ASTNode buildProgram(const ParseNode &node) const;
        ASTNode buildDeclarationPart(const ParseNode &node)
const;
        ASTNode buildConstDeclaration(const ParseNode &node)
const;
        ASTNode buildTypeDeclaration(const ParseNode &node)
const;
        ASTNode buildVarDeclaration(const ParseNode &node)
const;
        ASTNode buildType(const ParseNode &node) const;
        ASTNode buildArrayType(const ParseNode &node) const;
        ASTNode buildRange(const ParseNode &node) const;
        ASTNode buildEnumerated(const ParseNode &node) const;
        ASTNode buildRecordType(const ParseNode &node) const;
        ASTNode buildFieldList(const ParseNode &node) const;
        ASTNode buildFieldPart(const ParseNode &node) const;
        ASTNode buildSubprogramDeclaration(const ParseNode
&node) const;
        ASTNode buildProcedureDeclaration(const ParseNode
&node) const;
        ASTNode buildFunctionDeclaration(const ParseNode
&node) const;
        ASTNode buildBlock(const ParseNode &node) const;
        ASTNode buildFormalParameterList(const ParseNode
```

```

&node) const;
    ASTNode buildParameterGroup(const ParseNode &node)
const;
    ASTNode buildCompoundStatement(const ParseNode &node)
const;
    ASTNode buildStatementList(const ParseNode &node)
const;
    ASTNode buildStatement(const ParseNode &node) const;
    ASTNode buildAssignmentStatement(const ParseNode
&node) const;
    ASTNode buildIfStatement(const ParseNode &node) const;
    ASTNode buildCaseStatement(const ParseNode &node)
const;
    std::vector<ASTNode> buildCaseBranches(const ParseNode
&node) const;
    ASTNode buildWhileStatement(const ParseNode &node)
const;
    ASTNode buildRepeatStatement(const ParseNode &node)
const;
    ASTNode buildForStatement(const ParseNode &node)
const;
    ASTNode buildProcedureOrFunctionCall(const ParseNode
&node, bool asFunction) const;
    ASTNode buildParameterList(const ParseNode &node)
const;
    ASTNode buildExpression(const ParseNode &node) const;
    ASTNode buildSimpleExpression(const ParseNode &node)
const;
    ASTNode buildTerm(const ParseNode &node) const;
    ASTNode buildFactor(const ParseNode &node) const;
    ASTNode buildVariable(const ParseNode &node) const;
    std::vector<ASTNode> buildIndexList(const ParseNode
&node) const;
    ASTNode buildConstant(const ParseNode &node) const;

    std::vector<std::string> collectIdentifierList(const
ParseNode &node) const;
    const ParseNode *firstChildWithLabel(const ParseNode
&node, const std::string &label) const;
    std::string terminalValue(const ParseNode &node)
const;
    std::string operatorText(const ParseNode &node) const;
    ASTNode terminalAsLiteralOrIdentifier(const ParseNode
&node) const;
    ASTNode makeIdentifier(const std::string &name) const;
    bool hasLabel(const ParseNode &node, const std::string
&label) const;
    bool isTerminal(const ParseNode &node, int tokenType)

```

```
const;
};

class ASTBuilderError : public std::runtime_error {
public:
    explicit ASTBuilderError(const std::string &message);
};
}
```

B. Decorated AST

a. Symbol Table

Symbol table dirancang sebagai struktur data utama pada tahap semantic analysis untuk menyimpan dan mengelola informasi semantik dari identifier yang muncul dalam program Arion. Informasi yang disimpan mencakup nama identifier, jenis objek, tipe data, scope, referensi ke struktur tipe lain, alamat relatif, nilai konstanta, serta status inisialisasi. Dengan adanya symbol table, semantic analyzer dapat melakukan pemeriksaan deklarasi, pencarian identifier, validasi scope, pengecekan tipe, dan pendeteksian kesalahan semantik seperti penggunaan identifier yang belum dideklarasikan atau deklarasi ulang dalam scope yang sama.

Rancangan symbol table terdiri dari tiga tabel utama, yaitu tab, btab, dan atab, serta satu struktur tambahan untuk mendeskripsikan tipe khusus seperti subrange dan enumerated. Tabel tab berperan sebagai tabel identifier utama. Setiap identifier yang dideklarasikan dalam program, seperti nama program, konstanta, tipe, variabel, parameter, prosedur, fungsi, dan field record, akan dicatat sebagai entri pada tabel ini. Setiap entri menyimpan informasi seperti nama identifier, jenis objek, tipe data, referensi ke tabel lain jika diperlukan, lexical level, alamat relatif, nilai konstanta, status inisialisasi, serta link ke identifier sebelumnya dalam scope yang sama. Sementara itu, btab digunakan untuk menyimpan informasi block atau scope, seperti global block, program utama, prosedur, fungsi, record, atau block anonim. Melalui btab, hubungan antar-scope dapat direpresentasikan secara hierarkis sehingga pencarian identifier dapat dilakukan dari scope terdalam menuju scope luar.

Tabel atab digunakan untuk menyimpan informasi tipe array. Setiap array memiliki informasi tipe indeks, tipe elemen, referensi tipe elemen jika elemen berupa tipe komposit, batas bawah, batas atas, ukuran elemen, dan ukuran total array. Selain itu, array juga menyimpan informasi hasil resolusi batas indeks, seperti nilai ordinal batas bawah dan batas atas. Dengan rancangan ini, semantic analyzer dapat memeriksa validitas tipe indeks array, memastikan batas indeks valid, serta menghitung ukuran total array berdasarkan jumlah elemen dan ukuran setiap elemen. Untuk tipe khusus, digunakan struktur deskriptor tipe yang menyimpan informasi subrange dan enumerated. Subrange menyimpan tipe dasar, batas bawah, batas atas, serta nilai ordinal batasnya, sedangkan enumerated menyimpan daftar nilai yang valid, batas ordinal awal dan akhir, serta ukuran tipe.

Pada tahap inisialisasi, symbol table terlebih dahulu dikosongkan, kemudian diisi dengan entri awal untuk identifier bawaan. Identifier bawaan tersebut mencakup reserved word, tipe dasar seperti integer, real, boolean, char, dan string, konstanta boolean true dan false, serta prosedur bawaan seperti readln, writeln, dan write. Setelah itu, global block dibuat sebagai scope awal program. Dengan mekanisme ini, semantic analyzer dapat langsung mengenali tipe dasar dan identifier bawaan tanpa perlu deklarasi eksplisit dari pengguna. Pengelolaan scope dilakukan dengan konsep stack block. Ketika semantic analyzer memasuki block baru, block tersebut dimasukkan sebagai scope aktif dan menyimpan referensi ke block induknya. Ketika keluar dari block, scope aktif dikembalikan ke block sebelumnya. Mekanisme ini membuat program dapat mendukung nested scope dan tetap menjaga aturan visibilitas identifier.

Proses deklarasi identifier dilakukan dengan memeriksa terlebih dahulu apakah identifier yang sama sudah ada pada scope aktif. Jika identifier sudah ada pada scope yang sama, maka deklarasi dianggap tidak valid karena terjadi redeclaration. Jika belum ada, entri baru dimasukkan ke tab, diberi lexical level sesuai scope aktif, dan dihubungkan dengan identifier sebelumnya dalam block yang sama. Setelah entri ditambahkan, block aktif pada btab diperbarui agar menunjuk ke identifier terakhir yang baru dideklarasikan. Untuk variabel, parameter, dan field record, symbol table juga

mengatur alamat relatif. Variabel menggunakan offset berdasarkan ukuran variabel lokal pada block aktif, parameter menggunakan offset berdasarkan ukuran parameter, sedangkan field record menggunakan offset berdasarkan ukuran field dalam record. Dengan demikian, symbol table tidak hanya menyimpan informasi nama dan tipe, tetapi juga menyediakan informasi awal yang dapat digunakan pada tahap kompilasi berikutnya.

Proses pencarian identifier dilakukan dari scope aktif menuju scope luar. Semantic analyzer terlebih dahulu mencari identifier pada block aktif dengan menelusuri linked list identifier dalam block tersebut. Jika tidak ditemukan, pencarian dilanjutkan ke parent block hingga mencapai global block. Jika identifier tetap tidak ditemukan, pencarian dilanjutkan pada identifier bawaan. Apabila identifier tidak ditemukan di seluruh scope maupun daftar predefined identifier, program menghasilkan semantic error berupa undeclared identifier. Rancangan symbol table juga mendukung tipe komposit, yaitu array, record, subrange, dan enumerated, dengan menyimpan detail tipe pada tabel pendukung yang sesuai. Selain itu, error handling pada symbol table dirancang agar kesalahan semantik dapat dilaporkan secara informatif, seperti deklarasi ulang, identifier tidak ditemukan, referensi tabel tidak valid, batas subrange atau array tidak valid, tipe indeks array yang tidak diperbolehkan, serta nilai enumerated yang duplikat. Secara keseluruhan, symbol table berfungsi sebagai pusat informasi semantik yang mendukung pengelolaan identifier, scope, validasi tipe, dan pemberian anotasi pada Decorated AST.

Berikut adalah struktur Symbol Table yang memuat tab, atab, serta btab yang dibuat pada src/arion/SymbolTable.hpp

```
enum class SymbolObjectKind {  
    Reserved,  
    Program,  
    Constant,  
    Type,  
    Variable,  
    Procedure,  
    Function,  
    Parameter,  
};
```

```
        Field,  
        Unknown  
    };  
  
    enum class TypeKind {  
        Unknown,  
        Void,  
        Integer,  
        Real,  
        Boolean,  
        Char,  
        String,  
        Subrange,  
        Array,  
        Record,  
        Enumerated  
    };  
  
    enum class BlockKind {  
        Global,  
        Program,  
        Procedure,  
        Function,  
        Record,  
        Anonymous  
    };  
  
    enum class TypeDescriptorKind {  
        Unknown,  
        Subrange,  
        Enumerated  
    };  
  
    struct TabEntry {  
        std::string identifier;  
        int link = 0;  
        SymbolObjectKind object = SymbolObjectKind::Unknown;  
        TypeKind type = TypeKind::Unknown;  
        int ref = 0;  
        bool normal = true;  
        int lexicalLevel = 0;  
        int address = 0;  
        std::string value;  
        bool initialized = false;  
    };  
  
    struct ATabEntry {  
        TypeKind indexType = TypeKind::Unknown;
```

```

        TypeKind elementType = TypeKind::Unknown;
        int elementRef = 0;
        std::string low;
        std::string high;
        int elementSize = 1;
        int size = 0;
        int indexRef = 0;
        bool boundsResolved = false;
        int lowOrdinal = 0;
        int highOrdinal = 0;
    };

    struct BTabEntry {
        std::string name;
        int parent = -1;
        int last = 0;
        int lastParameter = 0;
        int parameterSize = 0;
        int variableSize = 0;
        int lexicalLevel = 0;
        BlockKind kind = BlockKind::Anonymous;
        TypeKind returnType = TypeKind::Void;
        int returnRef = 0;
        int ownerTabIndex = -1;
    };

    struct TypeDescriptor {
        TypeDescriptorKind kind = TypeDescriptorKind::Unknown;
        TypeKind baseType = TypeKind::Unknown;
        int baseRef = 0;
        std::string low;
        std::string high;
        bool boundsResolved = false;
        int lowOrdinal = 0;
        int highOrdinal = 0;
        std::vector<std::string> values;
        int size = 1;
    };

```

b. Pembuatan *Decorated AST*

Decorated AST dibuat sebagai kelas yang menerima input AST pada *constructor*-nya. Dalam kelas *DecoratedAST*, terdapat AST dan juga *symbol table*. Pendekorasian AST dilakukan dengan cara menelusuri setiap node pada AST dengan

algoritma Depth First Search secara rekursif dengan menggunakan *visit function*. Berdasarkan jenis dari node AST yang sedang ditelusuri, symbol table diisi dan node AST diberikan anotasi yang sesuai.

Pendekorasian AST dilakukan dengan asumsi bahwa sintaks sudah benar. Nilai-nilai yang dibutuhkan untuk pendekorasian diasumsikan dari apa yang diberikan pada ASTBuilder, baik *role* dari child node atau atribut yang ada pada node yang ditelusuri.

Pertama, node "Program" dideklarasikan dalam *symbol table*. Untuk node bertipe *ConstDeclaration*, nilai yang bisa digunakan hanya nilai bertipe dasar dan nilai dari *identifier* lain yang telah didefinisikan. Pada node yang bertipe *TypeDeclaration*, tipe dideklarasikan sesuai tipe yang diinginkan dengan menggunakan fungsi *declareType*, *declareArrayType*, *declareEnumeratedType*, *declareSubrangeType*, dan *declareRecordType* yang disediakan oleh *symbol table*. Pada node bertipe *VarDeclaration*, variabel dideklarasikan dengan tipe yang telah ada atau membuat tipe anonim tanpa nama (bisa tipe apa pun). Untuk node bertipe *Parameter*, parameter dideklarasikan dengan tipe yang telah ada atau dapat berupa tipe array anonim. Tipe tanpa nama akan diberikan nama "_anonymousTypeX" dengan X adalah jumlah tipe anonim yang telah dibuat. Karakter pertama adalah garis bawah supaya tidak bertabrakan dengan identifier lain.

Pada pendekorasian AST, *type compatibility* pun dicek. *Type compatibility* dibuat berdasarkan tipe data apa bisa dibuat menjadi tipe data apa. Berikut tabel *type compatibility* yang perlu diketahui:

Dari	Ke	Compatible	Keterangan
Integer	Real	V	Integer adalah subset dari real.
Real	Integer	V	Dibulatkan dengan menghapus bagian angka di belakang koma.
Integer	Boolean	X	Boolean dipisahkan dari tipe integer seperti pada Java

Boolean	Integer	X	Boolean dipisahkan dari tipe integer seperti pada Java
Integer	Char	X	Char dipisahkan dari tipe integer seperti pada Java.
Char	Integer	X	Char dipisahkan dari tipe integer seperti pada Java.
Char	String	V	Char bisa dianggap sebagai string dengan panjang 1.
String	Char	X	String bisa jadi hanya memiliki panjang lebih dari 1.
Integer	Enumerated	X	Enum tidak direpresentasikan dengan integer.
Enumerated	Integer	X	Enum tidak direpresentasikan dengan integer.
Array	Array	Bersyarat	Dua array <i>compatible</i> jika dan hanya jika tipe elemen kedua array sama dan subrange indeks dari kedua array <i>compatible</i>
Subrange	Subrange	Bersyarat	Dua subrange <i>compatible</i> jika dan hanya jika tipe dari indeksinya sama dan memiliki nilai subrange yang sama juga.
Enumerated	Enumerated	Bersyarat	Dua enumerated <i>compatible</i> jika dan hanya jika memiliki referensi tab ke indeks yang sama.
Record	Record	Bersyarat	Dua record <i>compatible</i> jika dan hanya jika memiliki referensi btab ke indeks yang sama.

Adapun tipe yang dibolehkan pada *unary operation* adalah sebagai berikut:

Operator	Tipe yang dibolehkan	Keterangan
not	Boolean	Operator boolean

+, -	Integer, Real	Hanya tipe numerik, operator sebagai penanda positif/negatif.
------	---------------	---

Adapun tipe yang dibolehkan pada *binary operation* adalah sebagai berikut:

Operator	Left Hand Side	Right Hand Side	Keterangan
+	Char, String	Char, String	Konkatenasi dua string
+, -, *	Integer, Real	Integer, Real	Operasi aritmetika dasar. Jika kedua operand adalah integer, hasil tetap integer
/	Integer, Real	Integer, Real	Operasi aritmetika dasar. Hasil pasti bilangan real
div, mod	Integer	Integer	Operasi aritmetika yang harus menggunakan bilangan bulat
==, <>	Semua tipe	Semua tipe	Semua tipe bisa dicek apakah sama dengan tipe yang sama pula.
>, >=, <, <=	Char, String, Integer, Real	Char, String, Integer, Real	Hanya tipe yang memiliki urutan yang jelas yang dapat dibandingkan.

Aturan *type compatibility* dipakai juga pada *statement* yang butuh *condition* berupa *boolean*, seperti pada *if*, *while*, *repeat-until*, dan *for statement*. Pada *statement* tersebut, ekspresi akan dicek apakah berupa *boolean*. Pada *case statement*, label pada tiap *branch case* harus sama persis dengan tipe ekspresi yang dicek. Pada *assignment statement*, ekspresi yang dihasilkan akan dicek apakah dapat dikonversi menjadi tipe pada variabel yang di-*assign*.

Block baru dibuat saat menelusuri node bertipe *Block*, *FunctionDeclaration*, dan *ProcedureDeclaration*, serta *if*, *while*, *repeat-until*, dan *for statement*. *Block* ini ditambahkan ke btab dengan fungsi *enterBlock* dari *symbol table*. *Block* akan menambah

level leksikal dari node yang didekorasi dalam *block* ini (secara rekursif) sekaligus menjadi pembatas *scope*. Setelah mendekorasi semua node anak dalam *block*, fungsi *leaveBlock* dipanggil untuk keluar dari *block* ini.

Parameter pada prosedur dan fungsi selalu di-*pass-by-value* karena tidak ada pembeda antara *pass-by-value* dan *pass-by-reference*.

Berikut adalah stuktur *Decorated Abstract Syntax Tree* yang memuat *visit function* yang telah dibuat pada `src/arion/DecoratedAST.hpp`:

```
class DecoratedAST {
private:
    ASTNode astTree_;
    SymbolTable symbolTable_;

    size_t anonymousTypeCount_ = 0;

    void dfs(ASTNode &astNode);
    void buildTableAndDecorateTree();

    bool decorateNode(ASTNode &node); // Return need to
    recursively decorate or no
    void decorateProgram(ASTNode &node);
    void decorateConstDeclaration(ASTNode &node);
    void decorateTypeDeclaration(ASTNode &node);
    void decorateFieldDeclaration(ASTNode &node, int
recordBlockRef);
    std::pair<int, TypeKind> decorateNamedType(ASTNode
&node);
    std::pair<int, TypeKind> decorateAnonymousType(ASTNode
&node);
    void decorateVarDeclaration(ASTNode &node);
    void decorateProcedureDeclaration(ASTNode &node);
    void decorateFunctionDeclaration(ASTNode &node);
    void decorateBlock(ASTNode &node, std::string name =
"block");
    void decorateParameter(ASTNode &node);
    void decorateAssignmentStatement(ASTNode &node);
    void decorateIfStatement(ASTNode &node);
    void decorateCaseStatement(ASTNode &node);
    void decorateWhileStatement(ASTNode &node);
    void decorateRepeatStatement(ASTNode &node);
```

```

        void decorateForStatement(ASTNode &node);
        void decorateProcedureCall(ASTNode &node);
        std::pair<int, TypeKind> decorateFunctionCall(ASTNode
&node);
        std::pair<int, TypeKind>
decorateBinaryOperator(ASTNode &node);
        std::pair<int, TypeKind> decorateUnaryOperator(ASTNode
&node);
        std::pair<std::string, TypeKind> decorateValue(ASTNode
&node);
        std::pair<int, TypeKind> decorateVariable(ASTNode
&node);
        std::pair<int, TypeKind> decorateArrayAccess(ASTNode
&node);
        std::pair<int, TypeKind> decorateFieldAccess(ASTNode
&node);
        std::pair<int, TypeKind> decorateExpression(ASTNode
&node);

        bool isAssignmentCompatible(int typeRef1, TypeKind
type1, int typeRef2, TypeKind type2);
        bool isTypeCompatible(int typeRef1, TypeKind type1,
int typeRef2, TypeKind type2);
        bool isOrderedType(TypeKind type);
        bool isNumberType(TypeKind type);
        bool isStringType(TypeKind type);
        bool isBooleanType(TypeKind type);
        bool isBuiltinWriteLikeProcedure(const TabEntry
&entry);
        bool isPrintableType(TypeKind type);

        void printTreeHelper(const ASTNode &node, int depth,
std::vector<bool> &isLast, std::ostream &out = std::cout,
ASTChildRole role = ASTChildRole::None) const;

    public:
        DecoratedAST(const ASTNode &astTree);

        const ASTNode &getASTTree() const;
        const SymbolTable &getSymbolTable() const;

        void printTable(std::ostream &out = std::cout) const;
        void printTree(std::ostream &out = std::cout) const;
};

class SemanticError : public std::runtime_error {
public:
    SemanticError(int line, std::string message);

```

<pre>} ; {</pre>

Pengujian

Kasus uji dapat dilihat di bagan berikut dan folder test/milestone-3/ di repositori GitHub

Tabel Hasil Pengujian	
1.	Input
	<pre> 1 program Hello; 2 3 var 4 a, b: integer; 5 6 begin 7 a := { Hello Saya Nafis } 5; 8 b := a + 10; 9 writeln('Result = ', b); 10 end.</pre>
	Output

1	tab:											
2	idx	identifier	object	type	ref	nrm	lvl	adr	link	value	initialized	
3												
4	0	<nil>	reserved	void	0	1	0	0	0		1	
5	1	and	reserved	unknown	0	1	0	0	0		1	
6	2	array	reserved	unknown	0	1	0	0	0		1	
7	3	begin	reserved	unknown	0	1	0	0	0		1	
8	4	case	reserved	unknown	0	1	0	0	0		1	
9	5	const	reserved	unknown	0	1	0	0	0		1	
10	6	div	reserved	unknown	0	1	0	0	0		1	
11	7	downto	reserved	unknown	0	1	0	0	0		1	
12	8	do	reserved	unknown	0	1	0	0	0		1	
13	9	else	reserved	unknown	0	1	0	0	0		1	
14	10	end	reserved	unknown	0	1	0	0	0		1	
15	11	for	reserved	unknown	0	1	0	0	0		1	
16	12	function	reserved	unknown	0	1	0	0	0		1	
17	13	if	reserved	unknown	0	1	0	0	0		1	
18	14	mod	reserved	unknown	0	1	0	0	0		1	
19	15	not	reserved	unknown	0	1	0	0	0		1	
20	16	of	reserved	unknown	0	1	0	0	0		1	
21	17	or	reserved	unknown	0	1	0	0	0		1	
22	18	procedure	reserved	unknown	0	1	0	0	0		1	
23	19	program	reserved	unknown	0	1	0	0	0		1	
24	20	record	reserved	unknown	0	1	0	0	0		1	
25	21	repeat	reserved	unknown	0	1	0	0	0		1	
26	22	integer	type	integer	0	1	0	0	0		1	
27	23	real	type	real	0	1	0	0	0		1	
28	24	boolean	type	boolean	0	1	0	0	0		1	
29	25	char	type	char	0	1	0	0	0		1	
30	26	string	type	string	0	1	0	0	0		1	
31	27	then	reserved	unknown	0	1	0	0	0		1	
32	28	to	reserved	unknown	0	1	0	0	0		1	
33	29	type	reserved	unknown	0	1	0	0	0		1	
34	30	until	reserved	unknown	0	1	0	0	0		1	
35	31	var	reserved	unknown	0	1	0	0	0		1	
36	32	while	reserved	unknown	0	1	0	0	0		1	
37	33	true	constant	boolean	0	1	0	1	0	true	1	
38	34	false	constant	boolean	0	1	0	0	0	false	1	
39	35	readln	procedure	void	0	1	0	0	0		1	
40	36	writeln	procedure	void	1	1	0	0	0		1	
41	37	<writeln-arg>	parameter	string	0	1	1	0	0		0	

42	38	write	procedure	void	2	1	0	0	0			1
43	39	<write-arg>	parameter	string	0	1	1	0	0			0
44	40	Hello	program	void	0	1	0	0	0			1
45	41	a	variable	integer	0	1	0	0	40			0
46	42	b	variable	integer	0	1	0	1	41			0
47												
48	atab:											
49	idx	idxType	idxRef	elType	elRef	low	high	resolved	lowOrd	highOrd	elSize	size
50	----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- -----											
51												
52	btab:											
53	idx	name	kind	parent	last	lpar	pSize	vSize	lvl	returnType	returnRef	owner
54	----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- -----											
55	0	Hello	program	-1	42	0	0	2	0	void	0	40
56	1	writeln	procedure	0	37	37	1	0	1	void	0	36
57	2	write	procedure	0	39	39	1	0	1	void	0	38
58												
59	type:											
60	idx	kind	baseType	baseRef	low	high	resolved	lowOrd	highOrd	size	values	
61	----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- -----											
62												
63	Program (name: Hello) -> (tabIndex: 40, lexicalLevel: 0)											
64	└─ Declarations [role: Declaration]											
65	└─ VarDeclarations [role: Declaration]											
66	└─ VarDeclaration [role: Variable] (name: a) -> (type: integer, tabIndex: 41, lexicalLevel: 0)											
67	└─ NamedType [role: Type] (name: integer) -> (type: integer, tabIndex: 22)											
68	└─ VarDeclaration [role: Variable] (name: b) -> (type: integer, tabIndex: 42, lexicalLevel: 0)											
69	└─ NamedType [role: Type] (name: integer) -> (type: integer, tabIndex: 22)											
70	└─ CompoundStatement [role: Body]											
71	└─ StatementList [role: Body]											
72	└─ Assignment [role: Statement]											
73	└─ Variable [role: Target] (name: a) -> (type: integer, tabIndex: 41, lexicalLevel: 0)											
74	└─ IntegerLiteral [role: Value] (value: 5)											
75	└─ Assignment [role: Statement]											
76	└─ Variable [role: Target] (name: b) -> (type: integer, tabIndex: 42, lexicalLevel: 0)											
77	└─ BinaryOperation [role: Value] (operator: +) -> (type: integer, lexicalLevel: 0)											
78	└─ Variable [role: Left] (name: a) -> (type: integer, tabIndex: 41, lexicalLevel: 0)											
79	└─ IntegerLiteral [role: Right] (value: 10)											
80	└─ ProcedureCall [role: Statement] (name: writeln) -> (type: void, tabIndex: 36, blockIndex: 1, lexicalLevel: 0)											
81	└─ Arguments [role: Arg]											
82	└─ StringLiteral [role: Arg] (value: 'Result = ')											
83	└─ Variable [role: Arg] (name: b) -> (type: integer, tabIndex: 42, lexicalLevel: 0)											

Hasil Uji Test Case 1:

Sesuai

2. Input

```
1  program SalahSemantik;
2
3  var
4      a: integer;
5      b: boolean;
6
7  begin
8      a := b
9  end.
```


	Output
	<pre>Input file path: test/milestone-3/input-9.txt Semantic error (line: 7): Incompatible assignable type: integer := boolean</pre>
Hasil Uji Test Case 2: Sesuai	
3.	Input
	<pre>1 program SemanticArrayType; 2 3 type 4 Numbers == array[1 .. 3] of integer; 5 6 var 7 arr: Numbers; 8 flag: boolean; 9 10 begin 11 flag := true; 12 arr[flag] := 1 13 end.</pre>
	Output
	<pre>Input file path: test/milestone-3/input-10.txt Semantic error (line: 12): Invalid array access of variable 'arr'. Invalid index type: boolean, expected integer</pre>
Hasil Uji Test Case 3: Sesuai	
4.	Input

	<pre> 1 program SemanticBinaryOperator; 2 3 var 4 a: integer; 5 s: string; 6 7 begin 8 s := 'hello'; 9 a := s + 1 10 end.</pre> Output Input file path: test/milestone-3/input-12.txt Semantic error (line: 9): Incompatible type in binary operation: string + integer
Hasil Uji Test Case 4:	
Sesuai	
5.	Input <pre> 1 program BentrekEnum; 2 3 ∨ type 4 Direction1 == (Left, Center, Right); 5 Direction2 == record dir : (Left, Center, Right); end; 6 7 ∨ var 8 dir1 : Direction1; 9 dir2 : Direction2; 10 11 ∨ begin 12 dir1 := Center; 13 dir2.dir := Center; 14 end.</pre> Output

	<pre>Input file path: test/milestone-3/input-15.txt Semantic error (line: 12): Incompatible assignable type: enumerated := enumerated</pre> Seharusnya, antara salah aja langsung di line 5 karena ada enum lain yang nilainya sama atau dibuat bisa aja sekalian.
Hasil Uji Test Case 5:	Belum Sesuai

Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan pada Milestone 3 tugas besar Arion Compiler, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Parse tree dari tahap *syntax analysis* berhasil dikonversi menjadi Abstract Syntax Tree (AST) yang lebih ringkas dan hanya menyimpan struktur semantik penting.
2. AST berhasil diproses menjadi Decorated AST (DAST) dengan tambahan annotation seperti tipe data, *lexical level*, indeks tab, atab, dan btab.
3. Program mampu melakukan berbagai pengecekan semantik, seperti deklarasi *identifier*, *redeclaration*, *assignment compatibility*, validasi ekspresi, akses *array*, serta pengecekan argumen *procedure/function*. Selain itu program juga dapat membedakan *syntax error* dan *semantic error*. *Syntax error* berhenti pada tahap *parser*, sedangkan *semantic error* terdeteksi setelah AST dibentuk dan diproses oleh Decorated AST.

B. Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan tahap berikutnya:

1. Membuat fungsi/prosedur bawaan untuk pengoperasian string dan konversi antar tipe data.
2. Membuat parameter supaya bisa dibedakan mana yang *pass-by-value* dan mana yang *pass-by-reference*. Pilihannya antara membuat token baru atau membuat tipe data tertentu untuk pasti di-*pass-by-reference*, misal *record*.
3. Memperbaiki penggunaan nilai enum yang dapat bentrok jika didefinisikan dalam level leksikal yang berbeda.

Lampiran

Link repository Github: <https://github.com/NafisKreatif/RGX-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No.	Nama	NIM	Pembagian Tugas	Kontribusi (%)
1	Wafiq Hibban Robbany	13524016	ASTNode, ASTBuilder	24
2	Muhammad Nafis Habibi	13524018	Decorated AST	28
3	An-Dafa Anza Avansyah	13524038	ASTBuilder	24
4	Wildan Abdurrahman Ghazali	13524054	Symbol Table	24

Referensi

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA: Pearson Addison-Wesley, 2006.